Pratica Giorno 5 — Gerarchie e contratti

🧪 Parte A — Laboratorio studenti

Implementa autonomamente la gerarchia `Persona` → `Studente` / `Docente`:

- **Persona**

  Contiene `nome`, `cognome`, `getNomeCompleto()` e `presentati()`

- **Studente extends Persona**

  Aggiunge `matricola`, usa `super()` nel costruttore, fa `@Override` di `presentati()`

- **Docente extends Persona**

  Aggiunge `materia`, usa `super()` nel costruttore, fa `@Override` di `presentati()`

- **Main con array Persona[]**

  Crea un array, popola con Studente e Docente, cicla e stampa `presentati()`

⚠️ Parte B — Errori controllati

Introduci e poi correggi ciascuno di questi errori:

super() dimenticato

Osserva l'errore del compilatore quando manca nel costruttore figlio

@Override con parametri diversi

Il compilatore segnala che non stai effettivamente ridefinendo nulla

new ClasseAstratta()

Tentare di istanziare direttamente `MetodoPagamento`

implements senza implementare

Dichiarare `implements Notificabile` senza scrivere `inviaNotifica()`

📝 Parte C — Esercizi assegnati

Esercizio 1 — Veicolo

Crea `Veicolo` (padre), `Auto` e `Moto` (figlie). Metodo `muoviti()` con override diverso. Array `Veicolo[]` e ciclo polimorfico.

Esercizio 2 — Scontabile

Crea interfaccia `Scontabile` con `double applicaSconto(double prezzo)`. Implementala in `ProdottoFisico` e `ProdottoDigitale` con logiche diverse.

Bonus — MetodoPagamento completo

Classe astratta `MetodoPagamento`, classi `PagamentoCarta` e `PagamentoPaypal`, interfaccia `Notificabile` su almeno una classe.

✅ Checklist finale del Giorno 5

- [x] ~~extends crea ereditarietà tra classi~~
- [x] ~~super richiama costruttore e metodi del padre~~
- [x] ~~@Override ridefinisce un comportamento ereditato~~
- [x] ~~Il polimorfismo usa un tipo comune con comportamenti diversi~~
- [x] ~~Le classi astratte modellano basi comuni incomplete~~
- [x] ~~Le interfacce modellano contratti (capacità)~~
- [x] ~~Una classe estende una sola classe, ma implementa più interfacce~~
- [x] ~~Questi concetti preparano a Spring, Service, Repository e DI~~

> Stai progettando classi rigide, o stai creando **tipi flessibili che possono evolvere**?